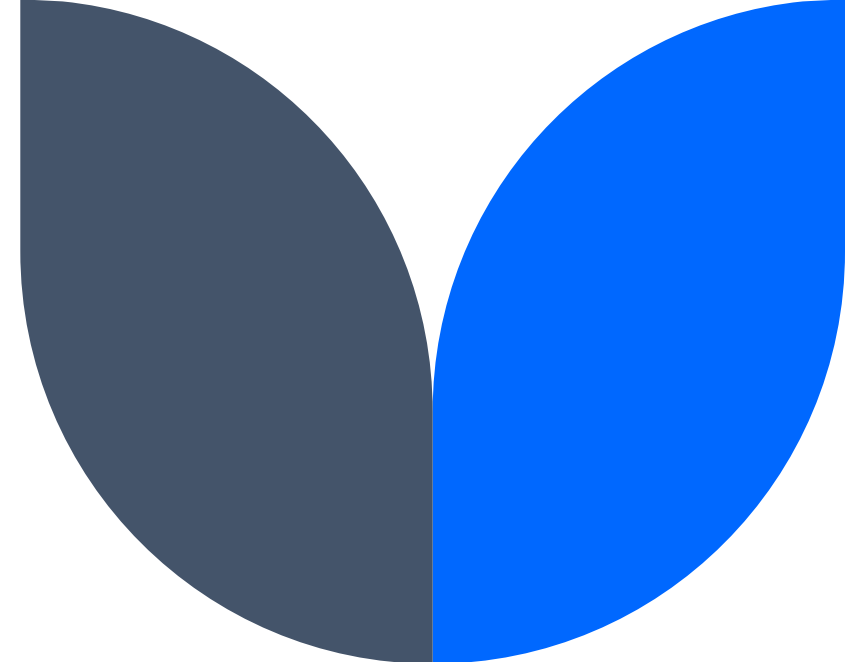
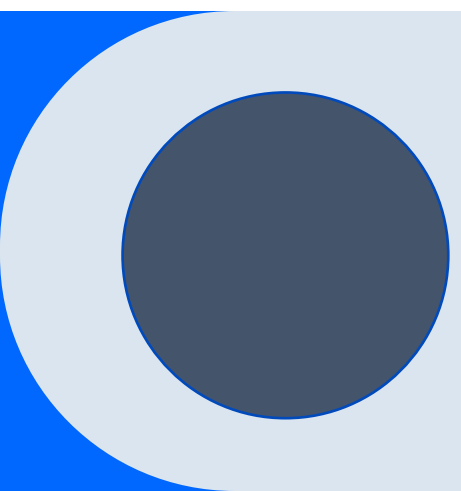




Многоуровневые модели предметной области. Модель «сущность-связь». Пример проектирования БД.

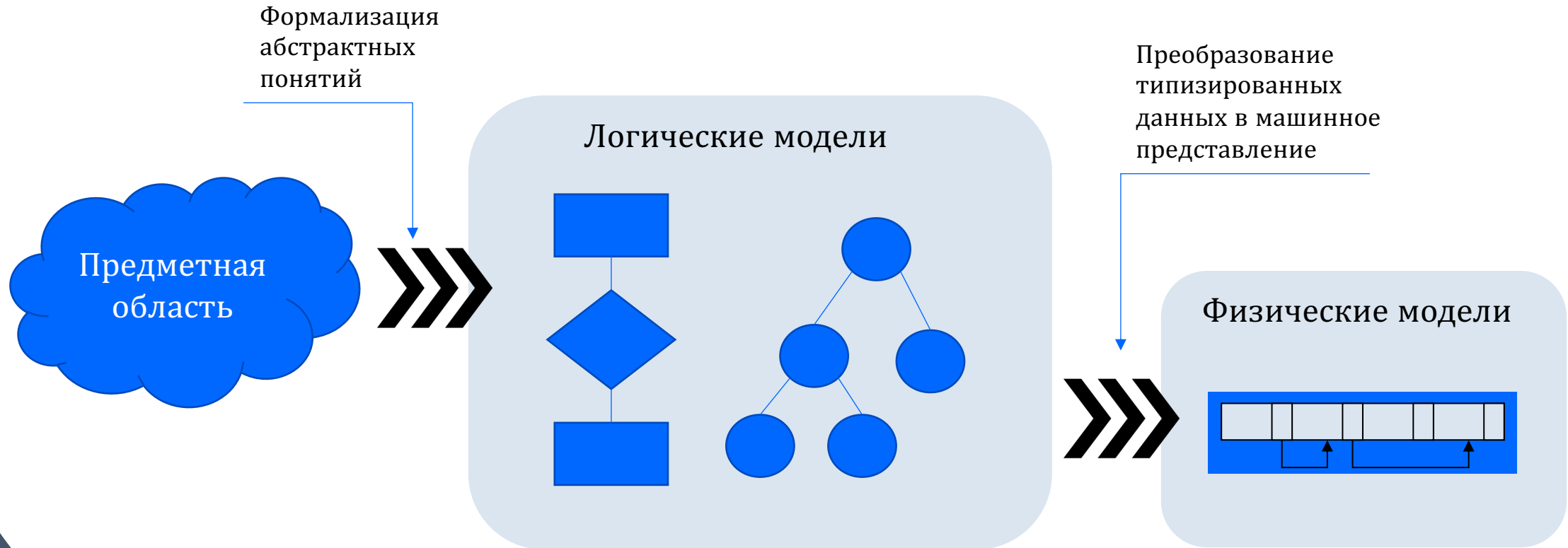
Валова Анастасия Александровна
НИТУ «МИСиС»



План

- Представления предметной области и модели данных
- Типология моделей представления информации
- Поэтапное проектирование базы данных
- Инфологические модели
- Типология связей и свойств объекта
- Графический язык модели «сущность-связь» в нотации IDEF1X
- Получение реляционной схемы из ER-диаграммы
- Подготовка к лабораторным работам № 5,6

Представления предметной области и модели данных



Многоуровневые модели предметной области.
Модель «сущность-связь». Пример проектирования БД.

Типология моделей представления информации

Концептуальный уровень

Инфологические модели

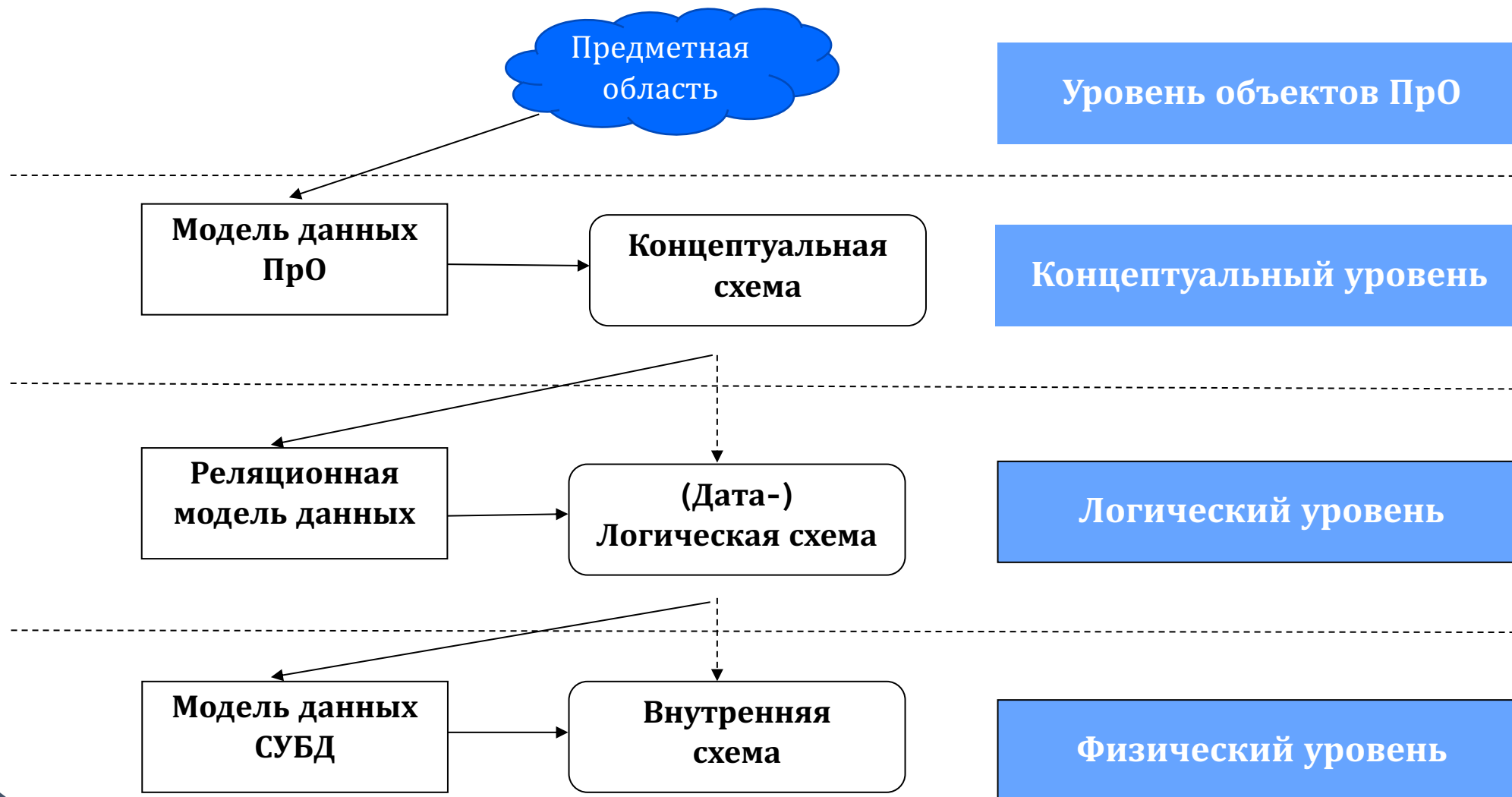
Логический уровень

Даталогические модели

Физический уровень

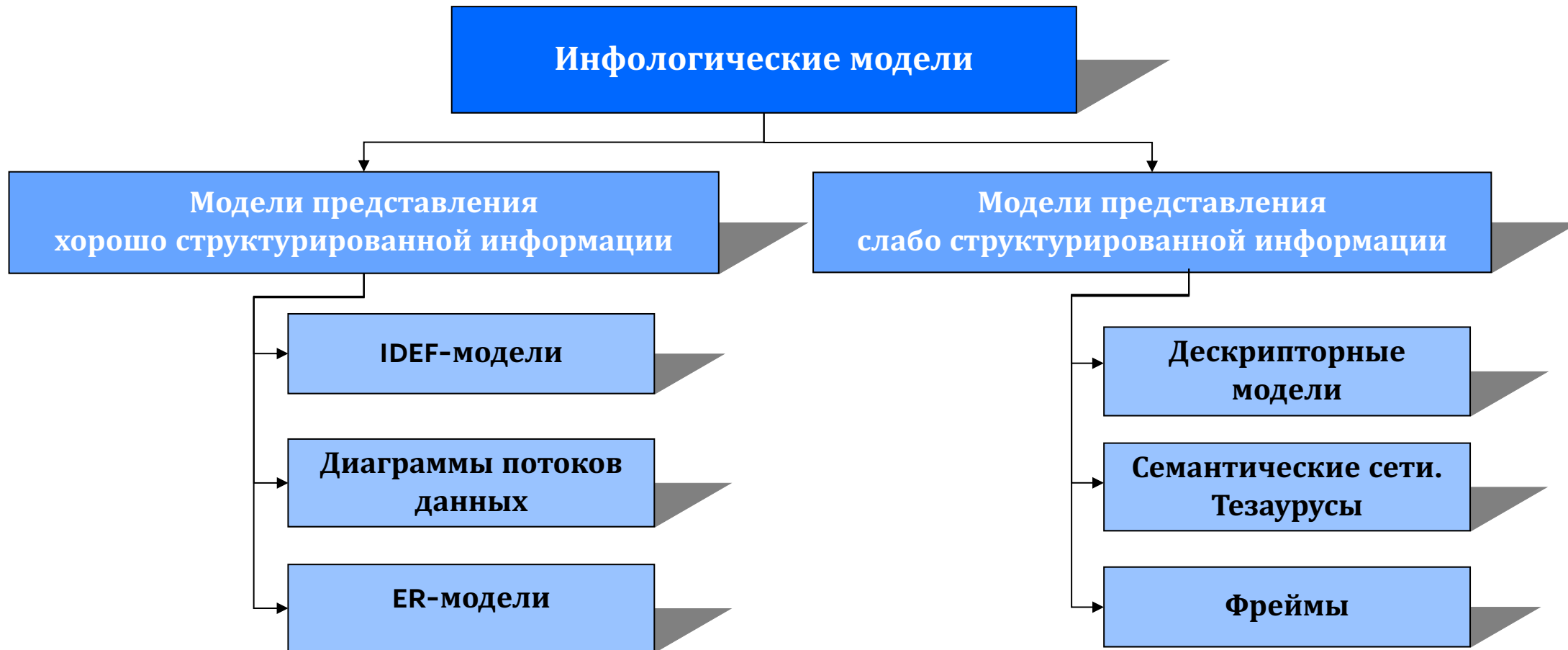
Физические модели

Поэтапное проектирование базы данных



Многоуровневые модели предметной области.
Модель «сущность-связь». Пример проектирования БД.

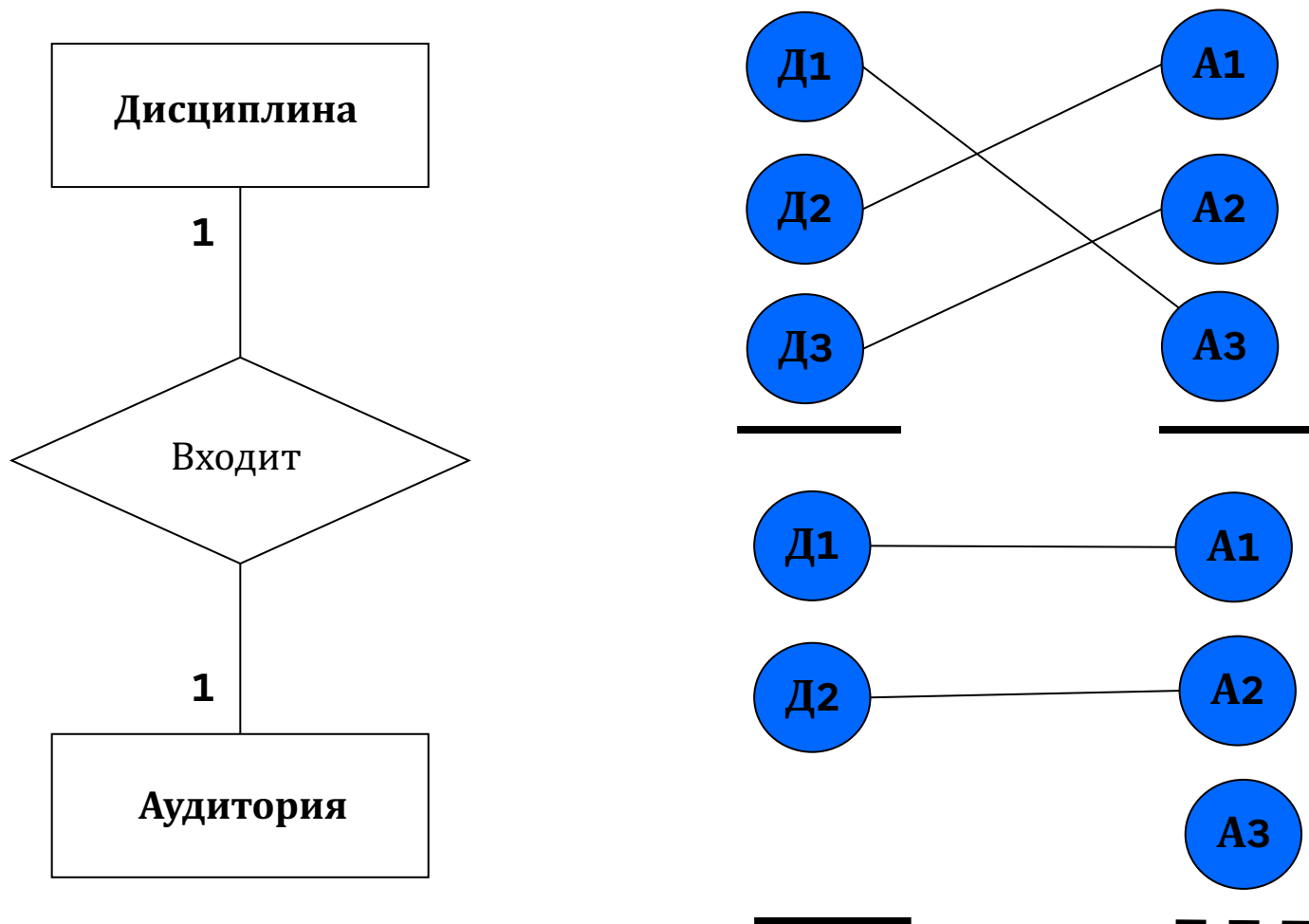
Инфологические модели



Многоуровневые модели предметной области.
Модель «сущность-связь». Пример проектирования БД.

Типология связей объекта

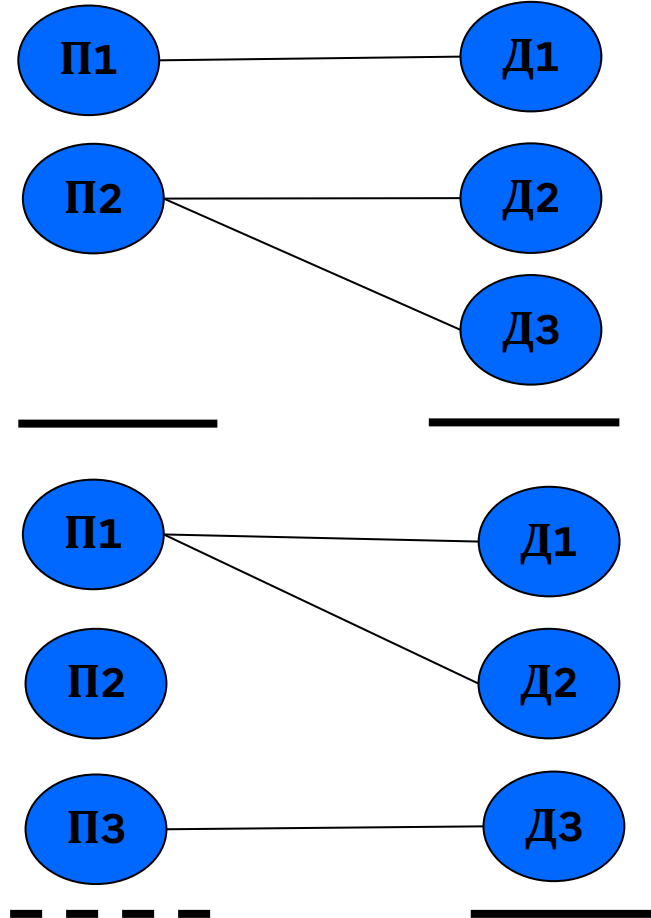
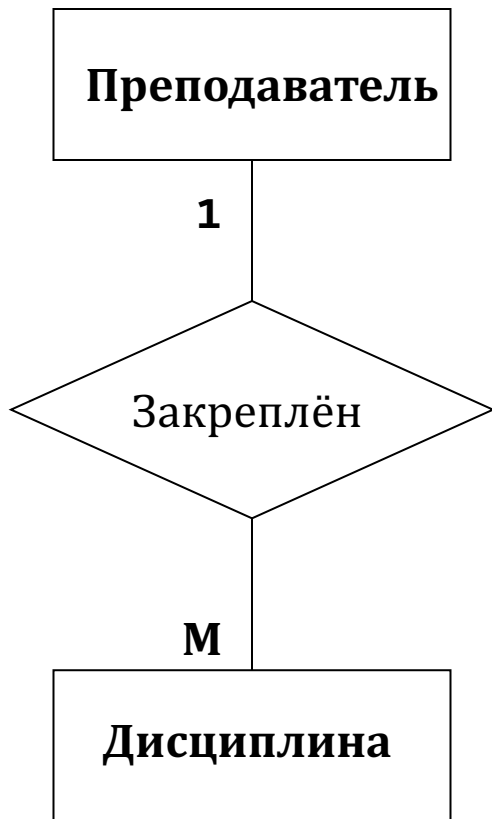
Связь мощности «один к одному» (1:1)



Типология связей объекта

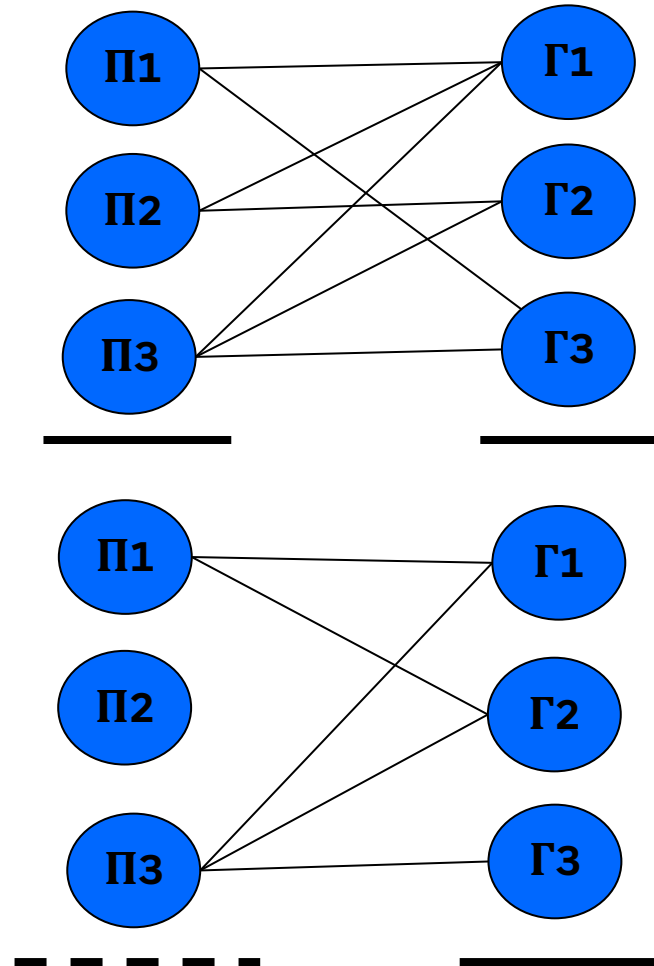
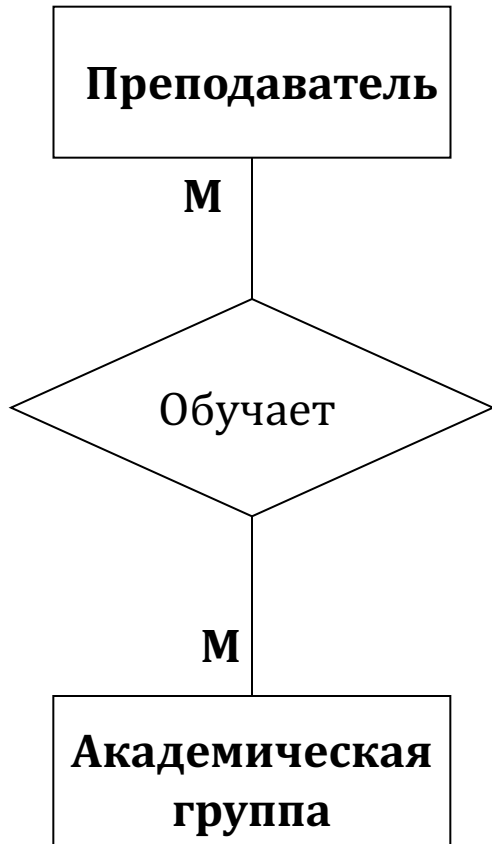
Связи мощности

«ОДИН КО МНОГИМ» (1:M) и «МНОГИЕ К ОДНОМУ» (M:1)



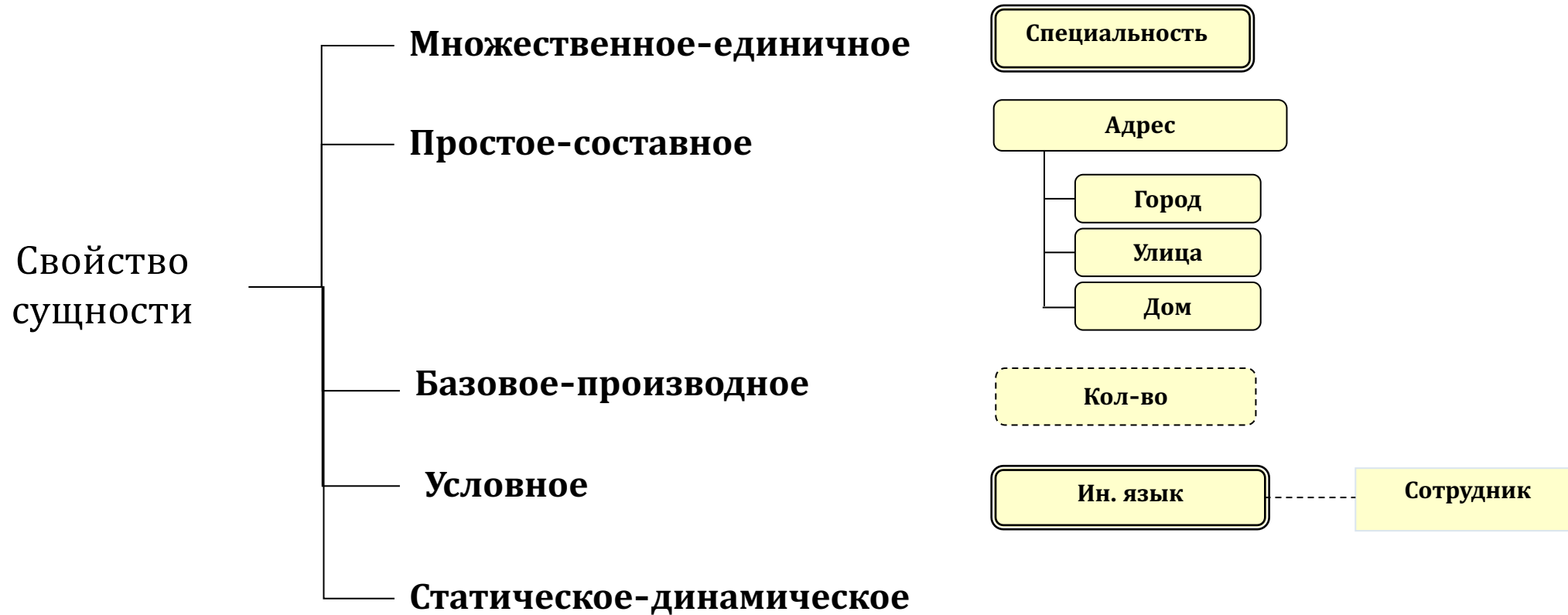
Типология связей объекта

Связь мощности «многие ко многим» (N:N)

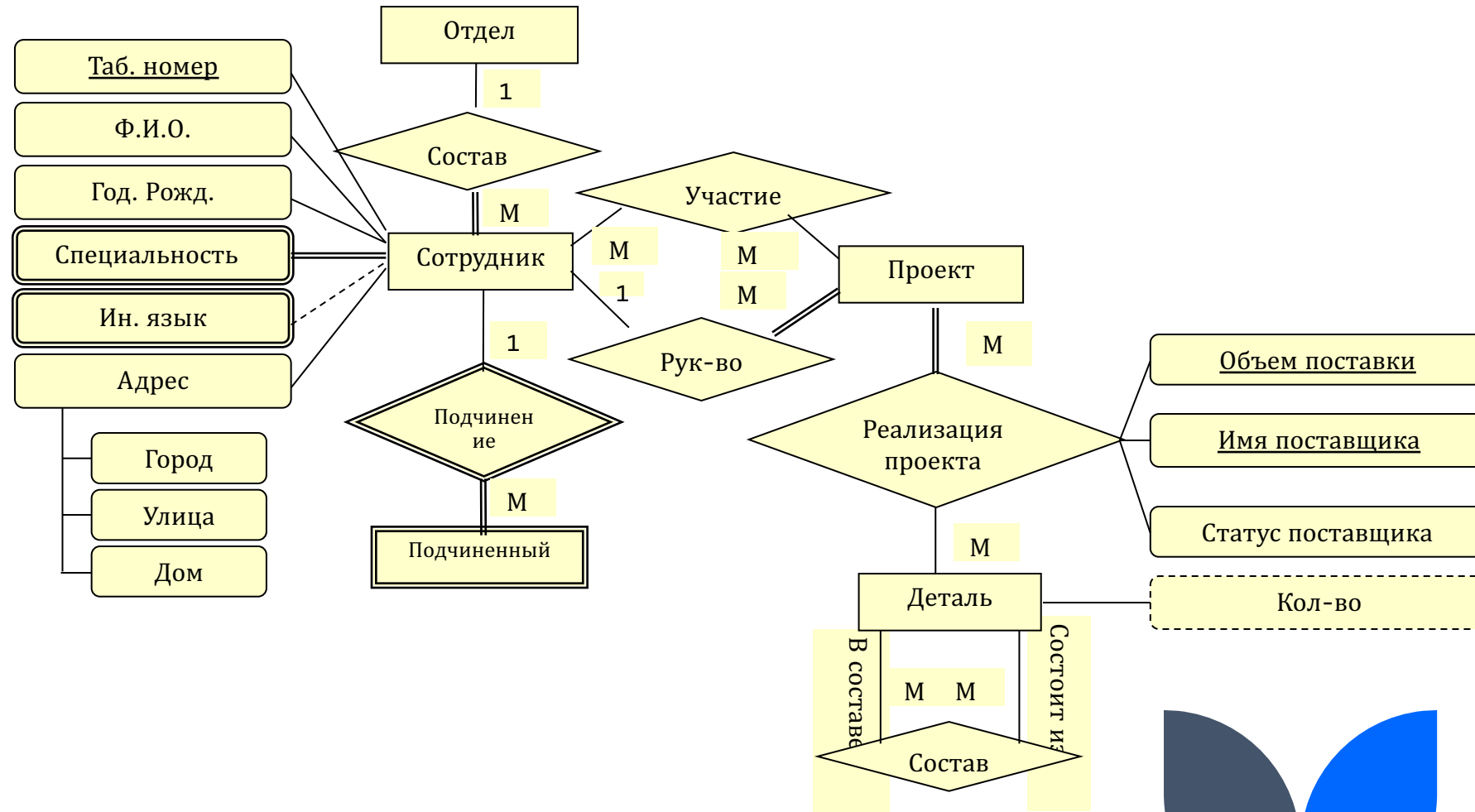


Многоуровневые модели предметной области.
Модель «сущность-связь». Пример проектирования БД.

Типология свойств объекта



ER-диаграмма предметной области сотрудники-проекты-детали-поставщики

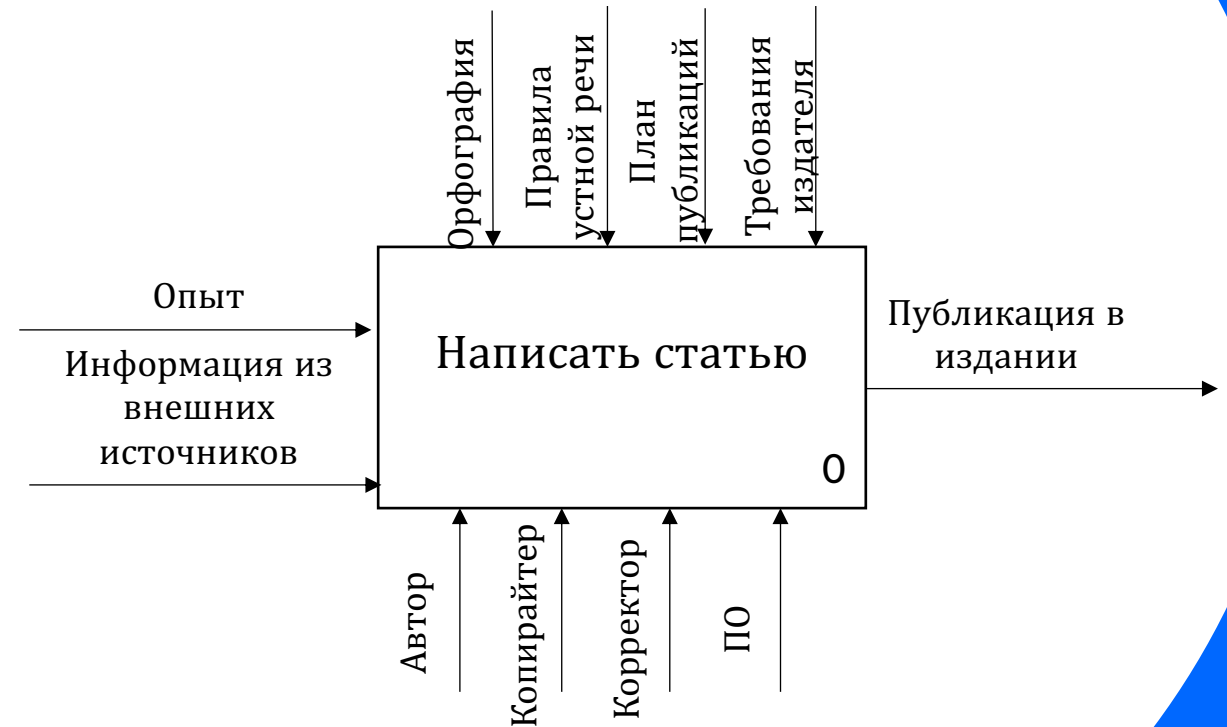


Многоуровневые модели предметной области.
Модель «сущность-связь». Пример проектирования БД.

Инфологические модели

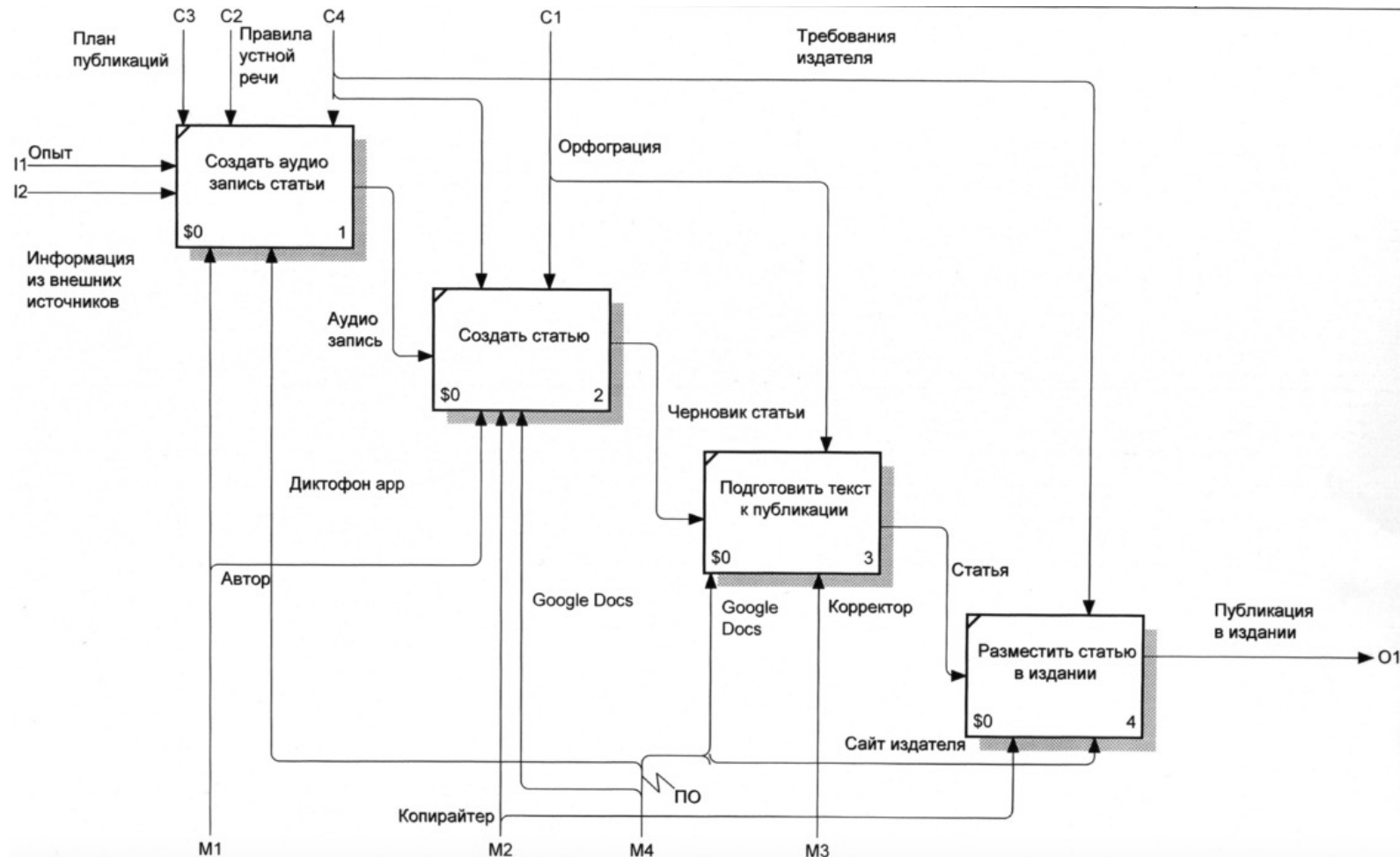
IDEF-модели

- Входящие – вводные, которые ставят определенную задачу.
- Исходящие – выводящие результат деятельности.
- Управляющие (сверху вниз) – механизмы управления (положения, инструкции и пр).
- Механизмы (снизу вверх) – что используется для того, чтобы произвести необходимую работу.



Инфологические модели

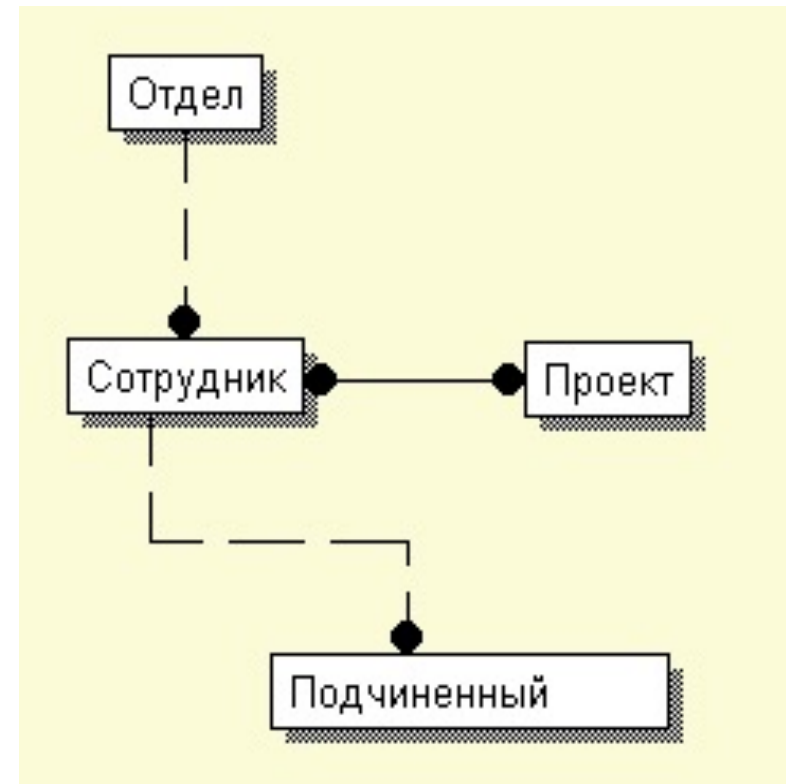
IDEF-модели



Многоуровневые модели предметной области.
Модель «сущность-связь». Пример проектирования БД.

Графический язык модели «сущность-связь» в нотации IDEF1X

Сотрудник ---	Сущность
Сотрудник Табельный номер Фамилия Имя Отчество Должность	Свойство сущности
_____	Связь типа «один к одному»
_____●	Связь типа «один ко многим»
●_____●	Связь типа «многие ко многим»



Фрагмент ER-диаграммы

Графические элементы основных нотаций

Нотация Элемент	Чена (Chen)	IDEF1x (IE)	Мартина (Martin)	Баркера (Barker)
Кардинальность 0, 1	Кардинальность связи указывается цифрами возле каждой сущности на линии, соединяющей ее со связью,			
Кардинальность строго N				
Кардинальность 0, 1...N				
Кардинальность 1...N				
Связь (не идентифицирую щая)			Имя связи указывается на линии, ее обозначающей	Сплошная линия.
Связь идентифицирую щая				Пунктирная линия.
Сущность независимая				
Сущность зависимая				
Атрибут		Список внутри прямоугольника сущности.	Список внутри прямоугольника сущности.	Список внутри прямоугольника сущности.
Первичный ключ		Записывается первым внутри сущности, отделяется от остальных сплошной линией	Подчеркивается	Отмечается символом # (решетка)

Получение реляционной схемы из ER-диаграммы

1. Каждая простая **сущность** превращается в **таблицу** (отношение). Имя сущности становится именем таблицы.
2. **Связь «многие ко многим»** рассматривается как сущность-связь и **превращается в таблицу** (отношение).
3. Каждый **атрибут** становится возможным **столбцом** с тем же именем.
4. Компоненты уникального идентификатора сущности превращаются в **первичный ключ**. Если имеется несколько возможных уникальных идентификатора, выбирается наиболее используемый.
5. Связи «многие к одному» и «один к одному» становятся внешними ключами. Т.е. делается копия уникального идентификатора с конца связи «один», и соответствующие столбцы составляют **внешний ключ**.
6. **Создаются индексы** для первичного ключа (уникальный индекс), а также внешних ключей и тех атрибутов, которые будут часто использоваться в запросах.

5 Использование функций и агрегация данных

Введение во встроенные функции (Built-In Functions)

Типы функций	Описание
Скалярные	Оперируют одним кортежем, возвращают единственное значение
Логические	Скалярные функции, сравнивающие множество значений, чтобы получить результат
Агрегатные	Принимают одно или несколько входных значений и возвращают одно значение
Оконные	Действуют на окне (набор кортежей)
Табличные	Возвращают виртуальную таблицу (набор кортежей), которая может быть использована в дальнейших SQL-запросах

Скалярные функции (scalar)

- Работают со входными данными из одной строки
- Возвращают одно (скалярное) значение
- Можно использовать в выражениях в запросах
- Может быть детерминированной или недетерминированной (deterministic / non-deterministic)

Типы скалярных функций

- Configuration
- Conversion
- Cursor
- Date and Time
- Mathematical
- Metadata
- Security
- String
- System
- System Statistical
- Text and Image

Логические функции

Выход определяется с помощью сравнительной логики

- ISNUMERIC

```
SELECT ISNUMERIC('101.99') AS Is_a_Number;
```

- IIF

```
SELECT productid, listprice, IIF(listprice > 50, 'high','low') AS  
PricePoint  
FROM Production.Product;
```

- CHOOSE

```
SELECT ProductName, Color, Size,  
       CHOOSE (ProductCategoryID,  
       'Bikes','Components','Clothing','Accessories') AS Category  
FROM Production.Product;
```

Оконные функции

```
OVER (  
    [ <PARTITION BY clause> ]  
    [ <ORDER BY clause> ]  
    [ <ROW or RANGE clause> ]  
)
```

```
<PARTITION BY clause> ::=  
PARTITION BY value_expression , ... [ n ]
```

```
<ORDER BY clause> ::=  
ORDER BY order_by_expression  
    [ COLLATE collation_name ] [ ASC | DESC ] [ ,...n ]
```

```
<ROW or RANGE clause> ::=  
{ ROWS | RANGE } <window frame extent>
```

```
<window frame extent> ::= {  
    <window frame preceding> | <window frame between>  
}
```

```
<window frame between> ::=  
    BETWEEN <window frame bound> AND <window frame  
bound>
```

```
-- все строки + PARTITION  
SELECT ProductCategoryID, ProductID, Name, ListPrice,  
AVG(ListPrice) OVER(PARTITION BY ProductCategoryID ORDER BY ListPrice asc ROWS BETWEEN  
    unbounded preceding and unbounded following) AS AvgPrice,  
Count(ListPrice) OVER(PARTITION BY ProductCategoryID ORDER BY ListPrice asc ROWS BETWEEN  
    unbounded preceding and unbounded following) AS CountPrice,  
Sum(ListPrice) OVER(PARTITION BY ProductCategoryID ORDER BY ListPrice asc ROWS BETWEEN  
    unbounded preceding and unbounded following) AS SumPrice  
FROM SalesLT.Product AS p  
ORDER BY ProductCategoryID asc;
```

Оконные функции

- Функции применяются к окну – набору строк
- Включают функции ранжирования, смещения, агрегации и распределения

```
SELECT TOP(3) ProductID, Name, ListPrice,  
       RANK() OVER(ORDER BY ListPrice DESC) AS RankByPrice  
FROM Production.Product  
ORDER BY RankByPrice;
```



ProductID	ProductName	UnitPrice	RankByPrice
8	Gizmo	263.50	1
29	Widget	123.79	2
9	Thingybob	97.00	3

Агрегатные функции

- Функции, которые оперируют на наборе или множестве строк
- «Обобщают» строки
- Без предложения GROUP BY, все строки рассматриваются как одна группа

```
SELECT COUNT(*) AS OrderLines,  
       SUM(OrderQty*UnitPrice) AS TotalSales  
FROM   Sales.OrderDetail;
```



OrderLines	TotalSales
542	71 4002.9136

Группировка с предложением GROUP BY

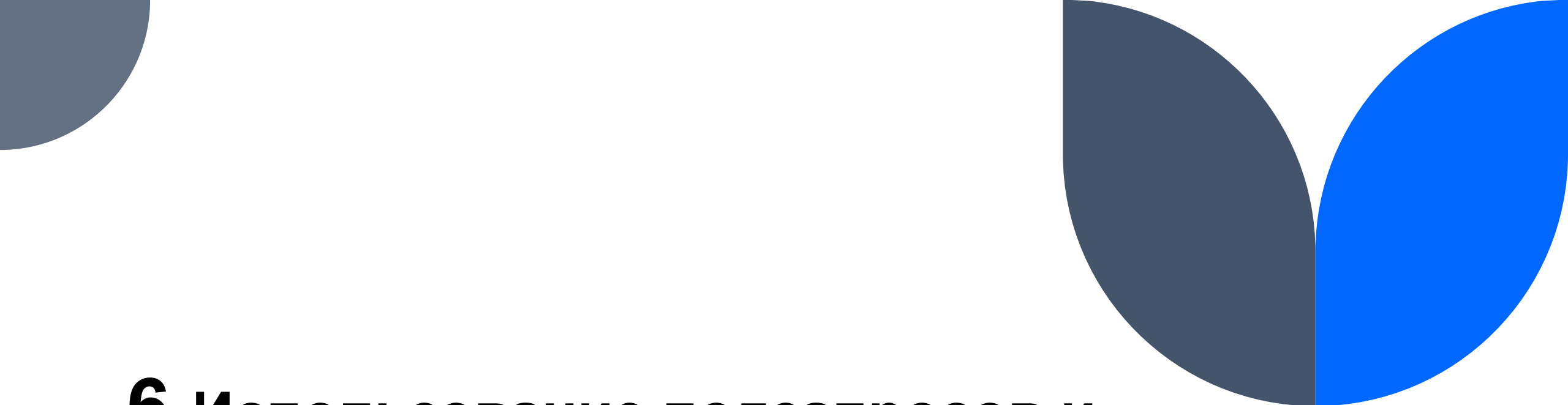
- GROUP BY создает группы для строк с результатом на основе уникальных комбинаций значений, определенных в предложении GROUP BY
- GROUP BY вычисляет обобщающие значения для агрегатных функций на последующих фазах
- Исходные строки с детализацией «теряются» после применения предложения GROUP BY

```
SELECT CustomerID, COUNT(*) AS Orders  
FROM Sales.SalesOrderHeader  
GROUP BY CustomerID;
```


Фильтрация с предложением HAVING

- Предложение HAVING накладывает условие, которому должна удовлетворять каждая группа
- Предложение WHERE обрабатывается перед GROUP BY, предложение HAVING обрабатывается после GROUP BY

```
SELECT CustomerID, COUNT(*) AS Orders  
FROM Sales.SalesOrderHeader  
GROUP BY CustomerID  
HAVING COUNT(*) > 10;
```



6 Использование подзапросов и предложения APPLY

Введение в подзапросы

- Подзапросы являются вложенными запросами: запросы в запросах
- Результаты вложенного запроса передаются внешнему запросу
 - Вложенный запрос выступает в роли выражения с точки зрения внешнего запроса



Скалярные и многозначные подзапросы

- Скалярный (однозначный) подзапрос возвращает одно значение во внешний запрос
 - Можно использовать везде, где можно использовать однозначные выражения: SELECT, WHERE и т.д.
- Многозначные подзапросы возвращают многозначные значения в виде одного столбца внешнему запросу
 - Используется с предикатом IN

```
SELECT orderid, productid, unitprice, qty  
FROM Sales.OrderDetails  
WHERE orderid =  
      (SELECT MAX(orderid) AS lastorder  
       FROM Sales.Orders);
```

```
SELECT custid, orderid  
FROM Sales.orders  
WHERE custid IN (  
      SELECT custid  
      FROM Sales.Customers  
      WHERE countryregion = N'Mexico');
```

Замкнутые и коррелированные подзапросы

- Большинство подзапросов замкнуты, т.е. не имеют взаимосвязи с внешним запросом помимо передачи результата
- Коррелированные подзапросы ссылаются на сущности (столбцы таблиц) внешнего запроса
 - Зависят от внешнего запроса, нельзя исполнять отдельно
 - Поведение подзапроса: один запуск для каждой строки внешнего запроса
 - Могут возвращать скалярное или множественное значение

```
SELECT orderid, empid, orderdate
FROM Sales.Orders AS O1
WHERE orderdate = (SELECT MAX(orderdate)
                  FROM Sales.Orders AS O2
                  WHERE O2.empid = O1.empid)
ORDER BY empid, orderdate;
```

Использование предложения APPLY с табличными функциями

- CROSS APPLY применяет табличное выражение справа к каждой строке в левой таблице
 - Концептуально похоже на CROSS JOIN между двумя таблицами, но может сопоставлять данные между источниками

```
SELECT S.supplierid, s.companyname, P.productid, P.productname, P.unitprice  
FROM Production.Suppliers AS S  
CROSS APPLY dbo.fn_TopProductsByShipper(S.supplierid) AS P
```

- OUTER APPLY добавляет строки со значениями NULL для правой таблицы
 - Концептуально похоже на LEFT OUTER JOIN между двумя таблицами